

```

        apply business rules
        status = "cancelled";
    }
}

```

A blunder in the above design is that the property ID is made mutable. Identities are never mutable. The *setId()* method is meaningless. The *Order*, being an entity, must be associated with any identity at the time of creation and must never be changed after that.

```

class Order {
    private int id;
    private double amount;
    private Date date;
    private String status;
    ....
    getId();
    getAmount();
    getDate();
    getStatus();

    setAmount(double);
    setDate(Date date);

    constructor() {
        generate and assign id;
    }

    constructor(int id) {
        assign id;
    }
    public void cancel() {
        apply business rules
        status = "cancelled";
    }
}

```

So far, so good. The next betterment is to replace the primitive types with value objects wherever they are appropriate. For instance, the above *Order* object accepts any double as the order amount. How is that acceptable? Isn't it reasonable to say that the order amount must be above zero? Let's refactor.

```

class Money {
    private double value;
    getValue();
    setValue(double);
}

```

```

class Order {
    private int id;
    private Money amount;
    ...
}

```

This looks a bit more reasonable. However, we again committed a blunder. Isn't *Money* a value object? Can a value object be modelled as mutable? No, right? Here is the correct version:

```

class Money {
    private double value;
    getValue();
    constructor(value) {
        validate the value;
        assign value;
    }
}

```

And, finally, a repository for *Order*:

```

interface OrderRepository {
    save(order)
    load(id);
    delete(id);
    loadAll();
    ...
}

```

There can be several implementations like *JpaOrderRepository*, *MongoOrderRepository*, *RedisOrderRepository*, and so on.

UMS as microservices

The user management system that we have been exploring in the last three parts of this series of articles is a single monolith system that offers multiple features. By applying the principles of domain-driven design, it can be decomposed into a set of microservices. And that is what we are going to see in the next part. **END** 🐼

By: Krishna Mohan Koyya

The author is the founder and the principal technology consultant at Glarimy Technology Services, Bengaluru. He has delivered more than 500 programs to corporate clients on design and architecture in the last 12 years. Prior to that, he held various positions in the industry and academia for a decade.